# `ComplexGitSync` 6.3.1– Architecture Overview

Nicolas Flipo

October 13, 2026

## Contents

**1 Architecture Overview** **1**

**2 Three-Tier Architecture** **1**

**3 Architectural Positioning** **1**

**4 Tier 1 — Core Data** **2**

**5 Tier 2 — Actions** **4**

**6 Tier 3 — Client / API** **5**

**7 Module Layout by Tier** **6**

**8 Document Hierarchy** **7**

**9 Registry Model** **8**

## 1 Architecture Overview

This chapter describes how `ComplexGitSync` is structured around three explicit tiers and explains the interaction between its classes. Related domain classes are grouped in modules according to the responsibility boundaries below.

## 2 Three-Tier Architecture

`ComplexGitSync` is organised into three tiers. Dependencies only flow **downward**: the Client calls Actions; Actions read and mutate Core Data; Core Data has no upward dependency.

Figure 1 shows the overall layout.

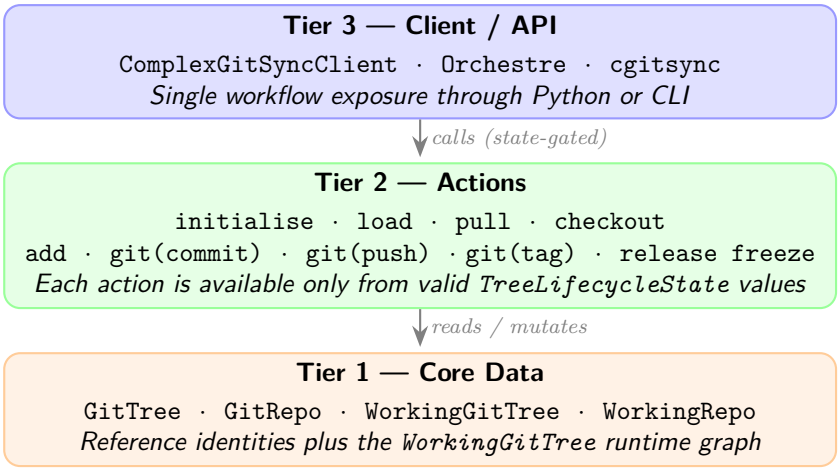
## 3 Architectural Positioning

`ComplexGitSync` is positioned as a deterministic synchronisation layer *on top of Git*, not as a replacement for Git itself. It combines two distinct graph scopes:

- **Git DAG**: per-repository commit history and native remote semantics.
- **GitTree DAG**: workspace-level dependency graph coordinating parent/root/leaf propagation and ordering.

The system therefore differs from:

Figure 1: Three-tier architecture: Client/API exposes state-gated methods; Actions operate on Core Data; Core Data separates reference identities from the `WorkingGitTree` parent–leaf runtime graph.




- plain Git usage (single repository scope),
- monorepo layouts (single storage boundary),
- submodule-only management (linking primitive without lifecycle orchestration),
- generic distributed sync engines (not constrained by Git-native '.gts'/'.lgr' deterministic contracts).

Figure 2 summarizes this positioning.

Figure 2: Architectural positioning matrix across Git scope and workspace coordination guarantees.

| Approach | Primary scope | What it does not guarantee |
|---|---|---|
| Git (single repo) | Commit DAG per repository | Multi-repository propagation and coordinated lifecycle gating |
| Monorepo | One repository, one commit DAG | Independent repository identities and per-repo remote ownership |
| Submodule management | Parent repo references child revisions | Deterministic tree-wide mutation workflow and snapshot contracts |
| ComplexGitSync | Git DAG + GitTree DAG | Not a credential broker or remote policy authority |

`release freeze` materializes deterministic checkpoints as `.gts` snapshots (schema + SHA-256 hash), and carries the project-scope State in the root repository's annotated tag, so `release list` and `release load` find a release from any workspace; `.lgr` assigns stable local ids and stores append-only ledger events for replayable local evolution.

Cross-declared nested repositories may initially resemble a local tangle at declaration time; runtime expansion normalizes this into a DAG via `fix_circularities`.

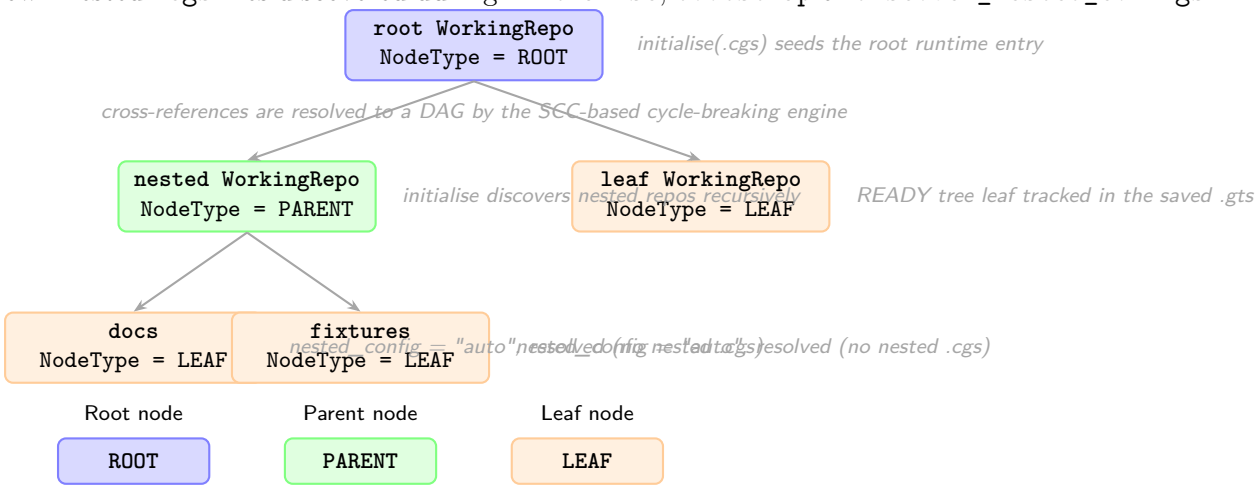
## 4 Tier 1 — Core Data

The reference model is centred on **`GitTree`**, which owns canonical **`GitRepo`** identities. Its runtime counterpart, **`WorkingGitTree`**, adds the parent–leaf graph of mutable **`WorkingRepo`** nodes.

A project consists of one *root* repository that may nest *parent* repositories and terminal *leaf* repositories.

Figure 3 shows a representative project tree.

Figure 3: `WorkingGitTree` is a parent–leaf graph of `WorkingRepo` nodes. Root and parent nodes own nested `.cgs` files discovered during `initialise`, `bootstrap` or `discover_nested_configs`.




Registry entries in `WorkingGitTree` are indexed by a colon-separated repo ID derived from their position in the tree (e.g. `root:deps/child-repo:docs`). Cross-referenced parents therefore create a tangle-like declaration layer, but the expanded runtime graph remains a DAG after the cycle-breaking engine has run.

## Cycle Breaking Engine

`fix_circularities` is a two-phase engine that converts a potentially cyclic dependency graph into a valid DAG:

1. **SCC detection (Tarjan's algorithm).** A directed dependency graph is built from the registry (edges: parent absolute path → child absolute path). `find_strongly_connected_components` identifies groups of paths that form a cycle. For each non-trivial SCC, an *Anchor* path is selected using three heuristics: (1) most external incoming edges, (2) closest to project root, (3) smallest SHA-256 hash. Back-edge entries — registry entries resolving to the anchor's path but parented inside a non-anchor SCC node — are flagged `is_external_reference = True` and removed.

2. **Hash-compatibility deduplication.** Residual path-duplicate entries are removed when their synchronisation state is compatible with the canonical entry.

The clone engine uses a **sync stack** (set of absolute paths already entered into the clone pipeline) to exclude entries whose path is already being processed, providing defence-in-depth against any residual back-references that escape the registry cleanup phase.

`topological_sort(registry)` returns entries in parent-first order (Kahn's BFS algorithm) for safe sequential clone/pull operations. Runtime `pull` uses the same three-level direction: `ROOT -> PARENT -> LEAF`; every repository — root, parent, and leaf alike — is a plain independent clone and is pulled in that same parent-first order, rather than through any submodule linkage.

Figure 6 shows the class overview across all three tiers, `GitTree`/`GitRepo` among them; the full current module-by-module Ring-level dependency graph lives in `docs/DevGuide/architecture.md`.

# 5 Tier 2 — Actions

Actions are operations that read or mutate the Core tier. **Every action is gated by the current `TreeLifecycleState`**: actions that require a `READY` tree will refuse to execute and log a gating-refusal event if the tree is not ready.

Figure 7 shows the full lifecycle state machine. The public CLI presents this lifecycle through `initialise`, `pull`, and the READY-state sync commands; lower-level load/expand/validate stages are internal steps of those workflows.

| Action | Minimum state | Produces |
|---|---|---|
| `initialise(.cgs)` | none | clone tree + **READY** |
| `initialise(.gts)` | none | loaded/restored **READY** |
| `pull(.cgs\|.gts)` | READY | resynchronised **READY** |
| `checkout(.gts)` | READY | **READY** |
| `add` | **READY only** | READY |
| `commit <msg>` | **READY only** | READY |
| `push` | **READY only** | READY + updated hash |
| `tag <name>` | **READY only** | READY + updated tag |
| `release freeze` | **READY only** | next `.gts` id |
| `release load` | **READY only** | READY, detached at the release |

For `.cgs` refs, repositories can be declared by branch or by tag. Format validation is static and does not resolve either reference remotely. The runtime layer selects the declared target (tag takes precedence when both are present) and checks its availability through `GitRunner`.

The authoring boundary is explicitly `PARSE -> NORMALIZE -> VALIDATE`. TOML parsing produces authoring data; normalization expands `provider:owner/repository` identifiers and fills `main` branch defaults, `ssh`, automatic nested discovery, and relative paths. Validation receives only the complete canonical `CgsDocument`. Authoring syntax is therefore not the internal representation.

File and CLI authoring converge at this boundary. The CLI collects `-project` and repeatable `-repo` values, then calls the non-interactive `ComplexGitSyncClient.configure()` Python API. Python callers can use the same method directly. The facade delegates to `CgsDocument`; neither it nor the CLI owns repository grammar or normalization. Loading an equivalent `.cgs` file and using either definition path produce semantically identical canonical documents. `discover -write` runs the same offline pipeline and serializes the result without entering Git runtime.

The boundary is also responsible for the reverse projection. `CgsDocument.to_git_tree()` creates the reference model and `CgsDocument.from_git_tree()` converts a reference or working tree back to canonical configuration. `GitTree.to_cgs()` remains only as a convenience delegate; it does not assemble repository tables or format TOML. The minimal serializer then removes deterministically reconstructible defaults. After the generated TOML is parsed and normalized again, its canonical representation must equal the one before serialization. Byte equality is not part of this contract.

Repository placement is parent-relative. At the top level the parent path is the project root; during nested discovery it is the repository whose `.cgs` is being expanded. Thus `../sibling` resolves to an existing sibling directory. Discovery compares normalized absolute paths and keeps an existing registry entry instead of inserting a duplicate.

Placement and traversal are independent. `nested_config = auto` searches the repository root for exactly one `*.cgs` — finding zero resolves the repository as a normal leaf, finding more than one raises immediately — `disabled` stops discovery through that reference, and a relative `.cgs` path selects an exact file inside the repository, failing if that named file does not exist. Explicit paths may not escape the repository root.

The textual `provider:owner/repository` grammar belongs exclusively to `cgs_format.py`. Its `parse_repo_id()` function owns syntax validation and splitting. Normalization uses that function, and CLI repository arguments must reuse it rather than introduce another parser.

The complete `.cgs` format pipeline is deterministic and offline. Parsing, normalization, validation, and serialization do not probe repositories or execute Git. Remote existence and branch/tag availability are resolved only after the validated document enters the runtime layer, through explicit `GitRunner` operations.

Provider behavior remains in `git_repo.py`: the `GitProvider` enum, canonical provider set, known-host map, and `RepoAddress` URL composer operate on already-separated identity fields. GitHub, GitLab, and Codeberg map respectively to `github.com`, `gitlab.com`, and `codeberg.org`. `custom` has no known-host entry and requires an explicit `gitprovider_url`; no fallback host is guessed.

Before any mutating workflow step (`commit`, `push`, `tag`, or `release freeze`), the action layer now runs a workspace preflight validator. It checks dirty worktrees, detached HEADs, missing remotes, branch divergence, unresolved merges, and stale recorded `commit_sha` values. Diagnostics are severity-based: warnings report actionable-but-allowed states, while blocking errors stop the operation. `tag` still requires a clean worktree and a non-existing tag.

### Formal `.gts` snapshot specification (T31)

`.gts` now carries explicit schema and deterministic hash metadata in the `[document]` table: `schema_version = "1.1"`, `hash_algorithm = "sha256"`, and `snapshot_hash = "<64-hex>"`.

The canonical digest is computed from a deterministic payload made of `project`, `tree_state`, and sorted `repo_state` records, excluding volatile generation metadata (`generated_at` and `command_origin`). Non-root entries require `parent_absolute_path`; READY repositories require `commit_sha`. This makes `.gts` a stable checkpoint primitive for repeatable workspace state comparison.

Each `repo_state` record also carries the repository's provider identity: `gitprovider`, `group_name`, and `gitprovider_url` (mirroring the same fields a `.cgs` repo entry carries — §?? "Local Authoring Spec"), plus `access_protocol`. `gitprovider`/`group_name`/ `gitprovider_url` are part of the canonical hashed payload: they say *which repository this is*, so two snapshots pointing at different hosts are not the same state. `access_protocol` is deliberately excluded from the hash — it records only how the repository was last reached (ssh or https), not what it is, and a tree cloned entirely over ssh must hash identically to the same tree cloned over https. A `.gts` written before this schema addition has no `gitprovider` field at all; every command that would otherwise derive a clone URL from it (only the missing-repository restore path — `push`/`pull` instead convert the repository's actual existing remote in place, never rebuilding it from identity) refuses with an explicit error instead of silently defaulting to GitHub.

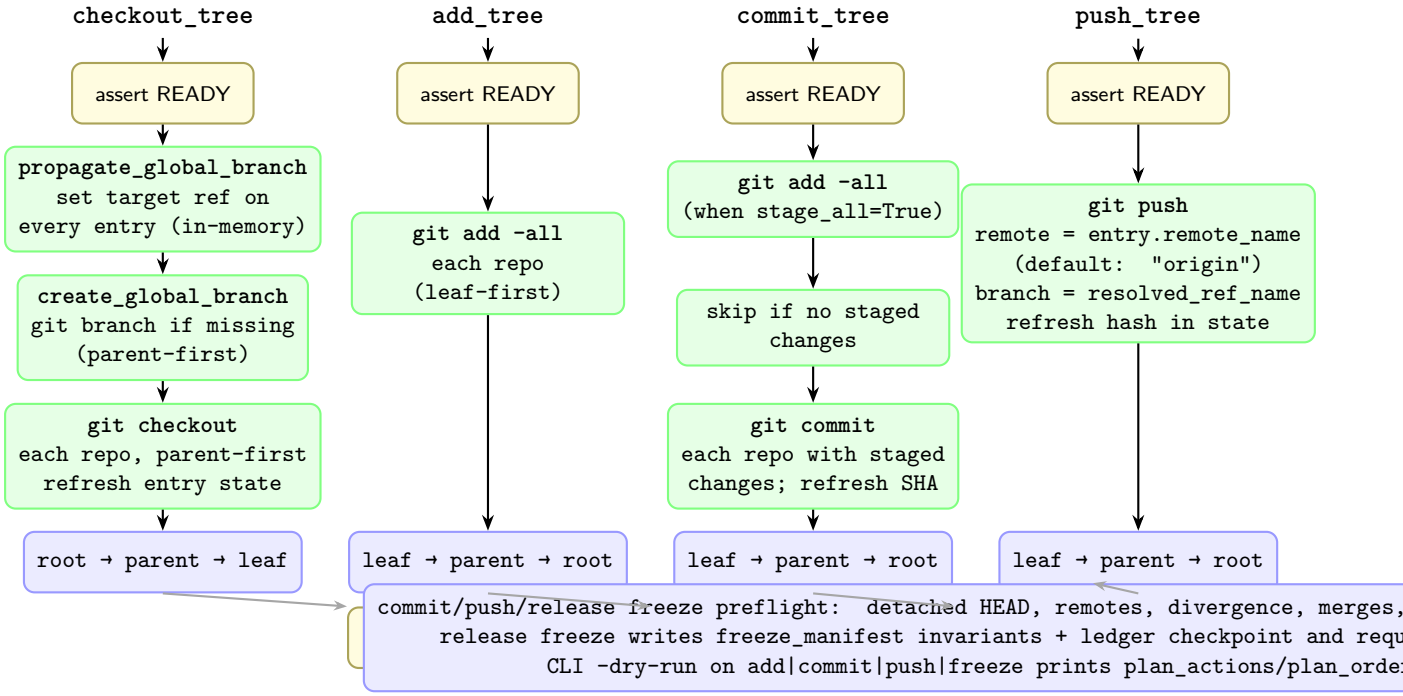
Figure 4 shows the internal sequence of the four tree-wide synchronisation actions implemented in the `operations/` package.

## 6 Tier 3 — Client / API

`ComplexGitSyncClient` is the single public facade. It:

- holds references to `Orchestre`, `GitRunner`, `RuntimeStateStore`, and the live `WorkingGitTree`;
- computes the current `TreeLifecycleState` on demand and gates every action against it;
- emits structured log events for every state transition, action start/end, fallback decision, and `.gts` write/load.

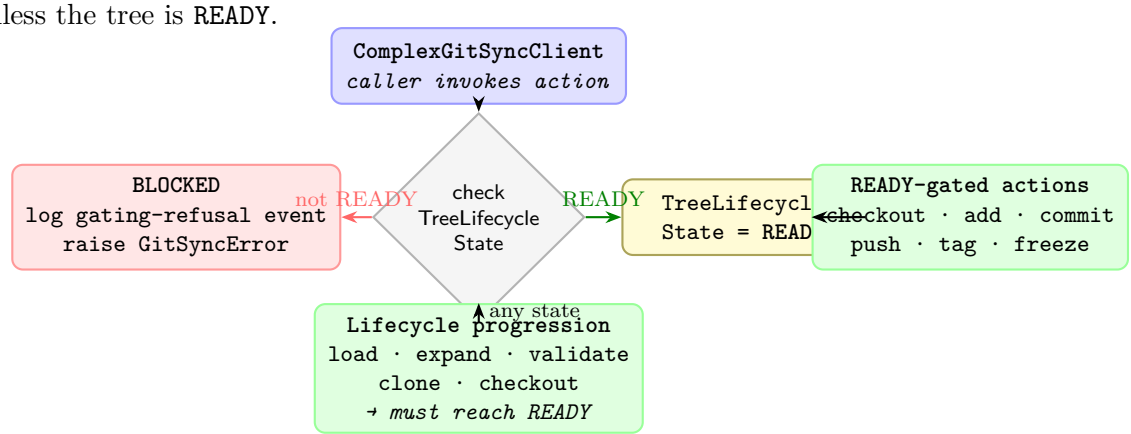
Figure 4: Synchronisation operations: `pull`/`checkout_tree` process `ROOT -> PARENT -> LEAF`; `add_tree`, `commit_tree`, and `push_tree` iterate `LEAF -> PARENT -> ROOT` so that inner repos are synchronized before their parents.




`Orchestre` is the coordination layer between the Client and the tree models. The CLI (`cli.py`) collects terminal arguments or interactive prompt values, then delegates to the reusable Client API. The Client delegates format semantics to `cgs_format.py`; runtime operations remain separate.

Figure 5 shows how the Client gates action access based on the live tree state.

Figure 5: Client state-gating: the Client reads `TreeLifecycleState` before delegating to an action. READY-gated actions (`checkout`, `add`, `commit`, `push`, `tag`, `release freeze`) are blocked unless the tree is `READY`.




## 7 Module Layout by Tier

Listing 1: Source modules grouped by architectural tier

```
src/ComplexGitSync/
  # --- Tier 1: Core Data (centred on GitTree / GitRepo) ---
  git_repo.py                    GitRepo, WorkingRepo, RepoAddress,
                                 RepoNode, enums
  git_tree.py                    GitTree, WorkingGitTree,
                                 ProjectTreeState, tree utilities,
                                 sync_gitignore (.gitignore maintenance)

  # --- Tier 2: Actions ---
  operations/                    BranchOperation, CommitOperation,
     MergeOperation,
                                 PushOperation, RestartOperation,
                                    RemovalOperation,
                                 Preflight, RepoOutcome
  orchestre/                     discovery and render collaborators
                                 (DiscoveryCommands, Reporting)

  # --- Tier 3: Client / API ---
  orchestre/                     ComplexGitSyncClient (facade) and the
     collaborators
                                 it delegates to; Orchestre (the tree
                                    holder)
  cli.py                         build_parser / main
  orchestre/                     RuntimeStateStore, CommandRunLogger
  master.py                      MasterConfig (workspace-local Git
     identity
                                 for ComplexGitSync-authored commits)

  # --- Document layer (cross-cutting) ---
  config_document.py             ConfigDocument (format-neutral base)
  cgs_format.py                  CgsDocument (.cgs parse/normalize/
     validate/serialize)
  gts_document.py                GtsDocument (runtime state snapshot)
  errors.py                      exception hierarchy
```

# 8 Document Hierarchy

Figure 8 shows the document class hierarchy.

All persistent documents share the `ConfigDocument` base class, which provides format-agnostic serialisation (`to_toml`, `to_json`, `to_yaml`) and factory methods (`from_toml`, `from_json`, `from_yaml`). Subclasses add document-specific validation and convenience properties. The base is defined in `config_document.py`; `cgs_format.py` owns the complete `.cgs`-specific boundary, while the `orchestre/` package consumes validated documents and retains runtime/orchestration responsibilities.

- **CgsDocument** (`.cgs`) — the authoring spec; describes the static project topology. *Never* a runtime snapshot.
- **GtsDocument** (`.gts`) — a generated snapshot capturing the exact state of the tree including commit SHAs and absolute paths. *Never* hand-edited.
- **Local Git Register** (`.lgr`) — the project-local register that assigns one stable local id to each generated `.gts` and tracks the current snapshot pointer.

# 9 Registry Model

Figure 9 shows the registry model.

`WorkingGitTree` is the authoritative in-memory graph. It maps *repo IDs* (colon-separated path strings such as `root:deps/child-repo`) to `WorkingRepo` instances. `WorkingRepo` holds both the static identity declared in `.cgs` and the dynamic state updated during operations.

The `RegistryTranslator` class in `registry.py` translates document objects into a live working tree:

```
# Load from authoring spec
registry = RegistryTranslator.from_cgs_document(document, config_path)

# Load from snapshot
registry = RegistryTranslator.from_gts_document(snapshot_document)

# Serialise to snapshot
gts_doc = RegistryTranslator.to_gts_document(
    registry, command_origin="clone", source_cgs_path=path
)
```

Figure 6 is deliberately kept at Tier granularity rather than diagramming all of `src/ComplexGitSync/`'s current modules — see `docs/DevGuide/architecture.md` for the dev-facing companion that carries the full per-module Ring-level breakdown and dependency graph this chapter does not attempt to reproduce.

Figure 6: Class overview by tier. Headline classes for each of the three tiers, plus the cross-cutting Document layer and the `cli/` adapter package that the Tier model never described. Every module in `src/ComplexGitSync/` is named here, either as a class box or in a tier's module list; see `docs/DevGuide/architecture.md` for the full current module-by-module Ring-level dependency graph.



Figure 7: GitTree lifecycle state machine. The simplified user-facing lifecycle is `initialise(.cgs)` → clone → `READY`; `initialise(.gts)` → restore → `READY`; `pull(.cgs|.gts)` → resync → `READY`. Synchronized Git operations (`checkout`, `add`, `git("commit")`, `git("push")`, `git("tag")`, `release freeze`) are gated on `READY`.

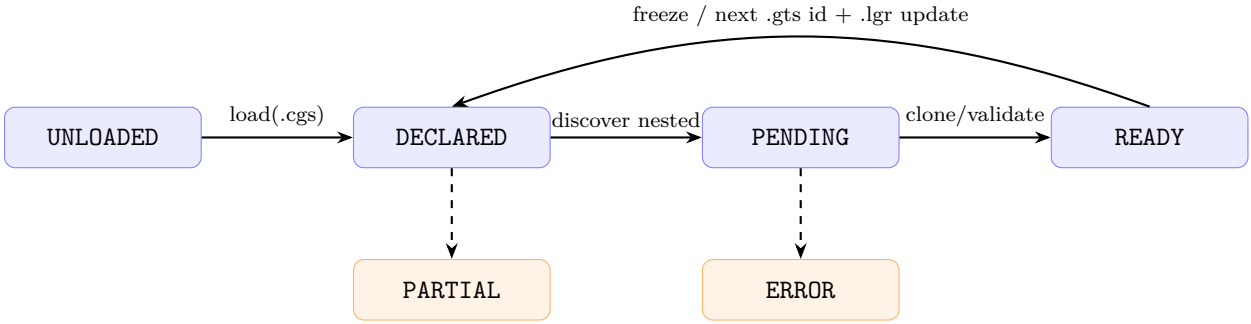



Figure 8: Document class hierarchy. `ConfigDocument` (`config_document.py`) is the abstract, I/O-free base; `ConfigDocumentIOMixin` (`config_document_io.py`) supplies the file-based `from/to_toml|json|yaml` methods. Each concrete document type combines both via ordinary multiple inheritance.

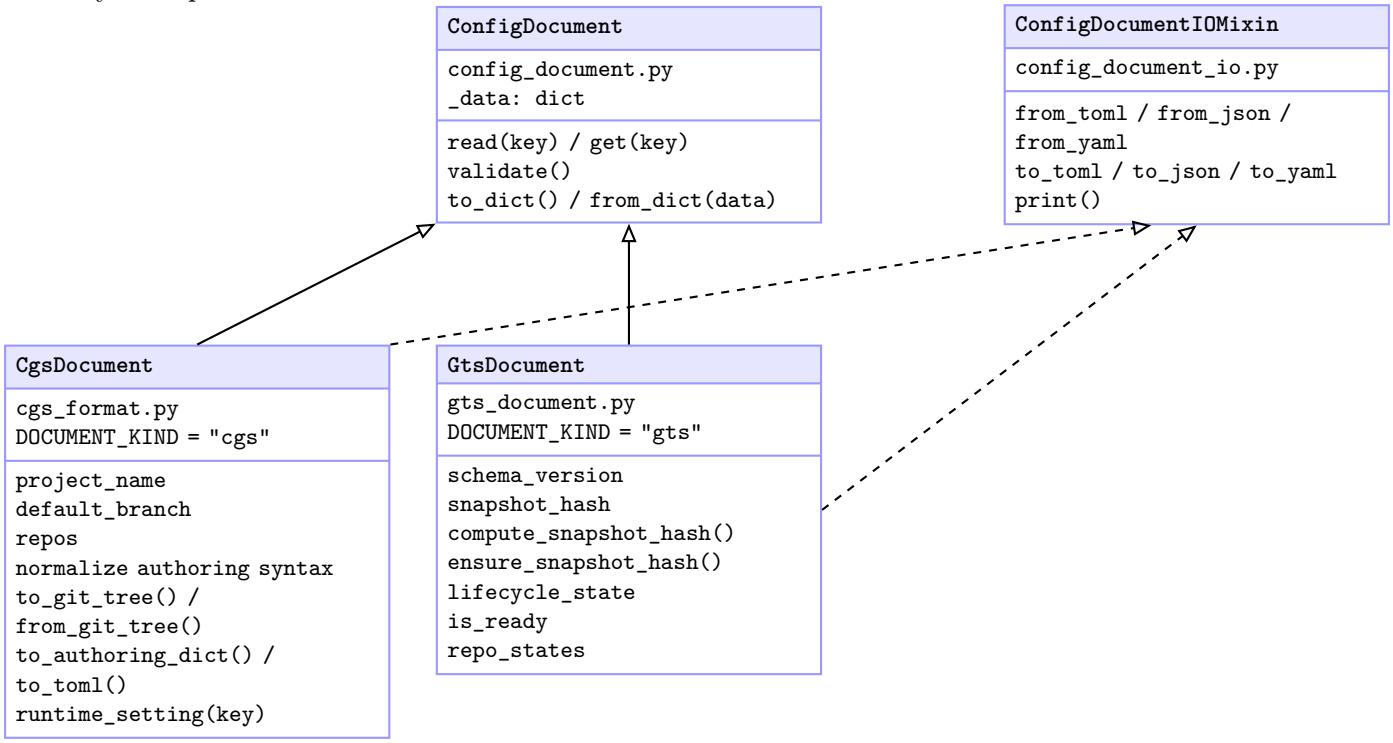


Figure 9: Registry model. `WorkingGitTree` is the authoritative in-memory graph; `WorkingRepo` *extends* `GitRepo` (Figure 6) rather than duplicating its identity fields, adding only the mutable tree-position and synchronisation state below.

